Understanding CockroachDB

A Concept Explainer for "The Resilient Geo-Distributed SQL Database" (SIGMOD 2020)

Sumit Kumar
22CS30056
CS60113: Advanced Database Systems

Contents

1 The Big Picture **2**

2 Core Concepts **2**
2.1 Sharding via Ranges . . . 2
2.2 Replication via Raft . . . 3
2.3 MVCC and Write Intents . . . 3
2.4 Serializability vs. Strict Serializability . . . 3

3 Contribution 1: Clock Synchronization Without Special Hardware **4**
3.1 Hybrid-Logical Clocks (HLC) . . . 4
3.2 Uncertainty Intervals . . . 4
3.3 Behavior Under Clock Skew . . . 4

4 Contribution 2: The Transaction Protocol **5**
4.1 Optimistic Concurrency with Timestamp Ordering . . . 5
4.2 Read Refreshes . . . 5
4.3 Write Pipelining and Parallel Commits . . . 6

5 Contribution 3: Controlling Where Data Lives **6**

6 The SQL Layer: Query Optimizer, Execution, and Schema Changes **7**
6.1 The Query Optimizer . . . 7
6.2 From Logical Plan to Physical Execution . . . 8
6.3 Two Execution Engines . . . 8
6.4 Online Schema Changes . . . 9

7 Results **9**

8 Case Studies **10**

9 Theoretical Comparison Table **11**

10 Glossary **11**

11 Frequently Asked Questions **12**

12 Conclusions **13**

1 The Big Picture

Imagine you run a company with customers in Europe, the United States, and Australia. You want one database that (a) keeps European customer data physically in Europe to satisfy GDPR, (b) lets Australian customers get fast responses without their queries crossing the ocean to a US datacenter, and (c) keeps working even if an entire datacenter burns down. A traditional relational database (think a single Postgres or MySQL server) simply cannot do all of this: it lives in one place, and if that place is unavailable, so is your data.

CockroachDB is a database designed from scratch to solve exactly this problem. It looks and feels like an ordinary SQL database to the application developer, but under the hood it spreads your data across many machines — potentially across continents — while still promising the same strong correctness guarantees (ACID transactions) that a single-machine database gives you.

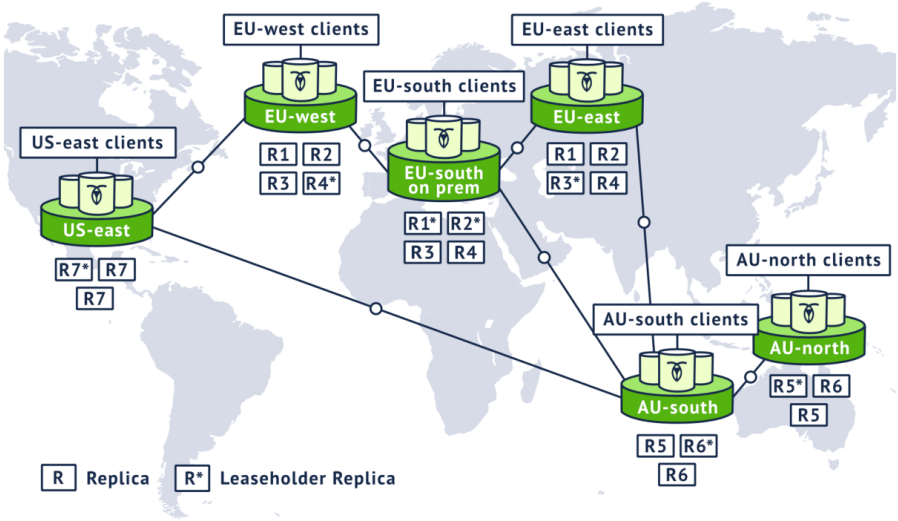

Figure 1 (paper): a global CockroachDB cluster. Each region holds local replicas (`R1`–`R7`); leaseholders (`R*`) are placed close to the clients they serve.

Insight: The central bet of the paper

You do *not* need exotic hardware (like Google's atomic clocks in Spanner) to get strong, serializable transactions across a globally distributed database. Ordinary commodity servers with loosely synchronized software clocks (like NTP) are enough, if you design the transaction protocol carefully.

The rest of this document builds up, piece by piece, exactly how CockroachDB pulls this off.

2 Core Concepts

Before we get to CockroachDB's specific contributions, we need shared vocabulary.

2.1 Sharding via Ranges

Definition: Range

A **Range** is a contiguous chunk (about 64 MiB) of the database's sorted key space. Every row in every table ultimately lives inside some Range. Ranges automatically split when they get too large and merge when they get too small.

Think of the entire database as one gigantic sorted phone book covering every row of every table. A Range is like one physical volume of that phone book — say, all entries from "Aaronson" to "Dawson." As more entries get added, that volume gets too fat, so it splits into two thinner volumes. This is what lets CockroachDB scale horizontally: just keep splitting and spreading volumes across more shelves (machines).

2.2 Replication via Raft

Definition: Raft

Raft is a consensus algorithm that lets a group of machines agree on an ordered sequence of operations, even if some machines fail, as long as a strict majority stay alive and reachable.

Each Range is replicated (by default, 3 copies on 3 different machines) as a *Raft group*. One replica is the leader; it proposes each write, and the write only "commits" once a majority of replicas (2 out of 3) have durably recorded it. This is like a 3-person committee: as long as 2 of the 3 are present and agree, decisions keep getting made, even if the third member is on vacation (or has crashed).

Definition: Leaseholder

Within a Raft group, one replica is designated the **leaseholder**. It is the only replica allowed to serve authoritative, up-to-date reads and to propose writes, avoiding an extra network round trip through Raft for every read.

2.3 MVCC and Write Intents

Definition: MVCC (Multi-Version Concurrency Control)

Instead of overwriting a value in place, MVCC stores every write as a new, timestamped version of that key. Readers can then pick exactly which version (i.e., which point in time) they want to see.

CockroachDB writes are not visible immediately as regular values — first they are written as **write intents**: provisional values tagged with a pointer to a *transaction record* (which says whether the transaction is `pending`, `staging`, `committed`, or `aborted`). A reader that stumbles on an intent looks up the transaction record to figure out whether to trust that value.

2.4 Serializability vs. Strict Serializability

Key Concept: Isolation Levels That Matter Here

Serializable isolation means that even though transactions run concurrently, the result is always *equivalent* to running them one at a time in *some* order. **Strict serializability** adds a stronger requirement: that "some order" must also match the real-world, wall-clock order in which transactions actually happened. CockroachDB guarantees serializability plus *single-key linearizability* (a slightly weaker real-time guarantee than full strict serializability) — this weaker guarantee is precisely what lets it avoid needing atomic clocks.

3 Contribution 1: Clock Synchronization Without Special Hardware

3.1 Hybrid-Logical Clocks (HLC)

The first problem: in a system spread across the world, how do different machines agree on the order of events, when their physical clocks are never perfectly in sync?

Definition: Hybrid-Logical Clock (HLC)

An HLC combines a machine's physical wall-clock time with a Lamport logical counter. Every message between nodes carries the sender's HLC value; the receiver updates its own clock to be at least as far ahead, ensuring causally related events always get increasing timestamps.

Analogy: imagine everyone in a large office synchronizes their wristwatch a little every time they shake hands with a colleague, nudging their watch forward if the other person's watch reads later. Over time, watches across the office naturally stay close together, and any two people who directly interacted will always have consistent before/after records of who did what first.

HLCs give three guarantees: **causality tracking** (if A causes B, B's timestamp is higher), **strict monotonicity** (a node's own clock never runs backwards, even across restarts), and **self-stabilization** (clocks that briefly drift apart converge again through normal communication).

3.2 Uncertainty Intervals

HLCs solve ordering *between machines that have talked to each other*. But what about two transactions on machines that never communicated directly? Their physical clocks could differ by up to some bound, called `max_offset` (defaulting to 500 ms).

Key Concept: Uncertainty Interval

Every transaction is assigned a provisional commit timestamp t and an uncertainty interval $[t,\ t + \text{max\_offset}]$. If, during its execution, the transaction reads a value whose timestamp falls inside this window, it cannot be sure whether that value was written before or after it in real time. Rather than guess, it performs an *uncertainty restart*: it pushes its own timestamp forward past the uncertain value and re-evaluates.

Analogy: suppose you're not sure if a friend's text message was sent right before or right after you made a phone call, because your phone's clock could be a few seconds off from theirs. Instead of guessing, you simply decide "let's treat it as happening after my call" and move on consistently. This sidesteps the need for a perfectly synchronized clock while still guaranteeing a consistent, real-time-compatible ordering for causally related operations.

Insight: Why this matters

This is the mechanism that replaces Spanner's TrueTime. Spanner *waits out* a tiny, hardware-guaranteed uncertainty window on every commit. CockroachDB instead *detects* and *resolves* uncertainty only when it actually causes a conflict, without needing hardware guarantees on how small that window is.

3.3 Behavior Under Clock Skew

What actually happens if two nodes' clocks drift *beyond* the configured `max_offset`? This matters because the whole scheme above assumes skew stays bounded.

Key Concept: Two Safeguards Against Clock Skew

Within a single Range, Raft's ordering never depends on clocks at all, so it stays correct regardless of skew. The risk is specifically at the *leaseholder* boundary — two nodes both believing they hold a valid lease could otherwise serve conflicting operations. CockroachDB prevents this with two safeguards: (1) lease intervals are constructed to be strictly disjoint in time, so an incoming leaseholder can never overlap with an outgoing one; (2) every Raft-replicated write carries the sequence number of the lease it was proposed under, and is rejected if that lease is no longer current.

Analogy: think of the leaseholder as holding a relay baton. The two safeguards guarantee the old runner has fully stopped before the new runner starts, and that any baton pass stamped with an old runner's number is refused — even if both runners' wristwatches disagree slightly about when the handoff happened.

Insight: The failure mode is graceful

Under severe clock skew, the absolute worst case is a stale read (a single-key linearizability violation) for a causally-dependent transaction issued through a different gateway node — *never* a broken transaction or inconsistent write. As an extra defense-in-depth measure, a node that detects its clock has drifted by more than 80% of the configured maximum offset from a majority of its peers will voluntarily self-terminate rather than risk serving incorrect reads.

4 Contribution 2: The Transaction Protocol

4.1 Optimistic Concurrency with Timestamp Ordering

CockroachDB transactions are essentially optimistic: they proceed without locking most data, and conflicts are resolved by *adjusting timestamps* rather than blocking.

- **Write-read conflict:** a read hitting an uncommitted intent waits for that transaction to finish.
- **Read-write conflict:** a write that would invalidate an already-served read is forced to advance its own commit timestamp past that read.
- **Write-write conflict:** a write colliding with a newer intent or committed value advances its timestamp similarly; true cycles are resolved via distributed deadlock detection.

4.2 Read Refreshes

Key Concept: Read Refresh

When a transaction's timestamp must move forward due to a conflict, CockroachDB first checks whether any key already read by that transaction changed value between the old and new timestamp. If nothing changed, the transaction can simply continue at the new timestamp — no restart needed.

Analogy: you've picked your groceries and are walking to checkout when the store says "prices just updated a minute ago." Instead of redoing your whole shopping trip, you just double-check that none of the items already in your cart changed price. If not, you check out as planned.

4.3 Write Pipelining and Parallel Commits

Two performance optimizations reduce the number of sequential network round trips a transaction needs.

Definition: Write Pipelining

Non-overlapping operations within a transaction can be sent for replication without waiting for earlier operations to finish replicating first — like an assembly line instead of one worker doing every step in strict sequence.

Definition: Parallel Commits

Normally, committing a transaction needs *two* sequential rounds of consensus: one to replicate the writes, one to replicate the "committed" marker. Parallel Commits introduces an intermediate `staging` status meaning "committed if all these writes finish replicating." This lets the commit marker replicate *at the same time as* the final writes, cutting a full round trip.

Insight: Measured impact

In the paper's microbenchmark, Parallel Commits improved throughput by up to **72%** and cut p50 latency by up to **47%**, and this benefit stayed constant even as the number of secondary indexes (and therefore the number of Ranges touched per transaction) increased.

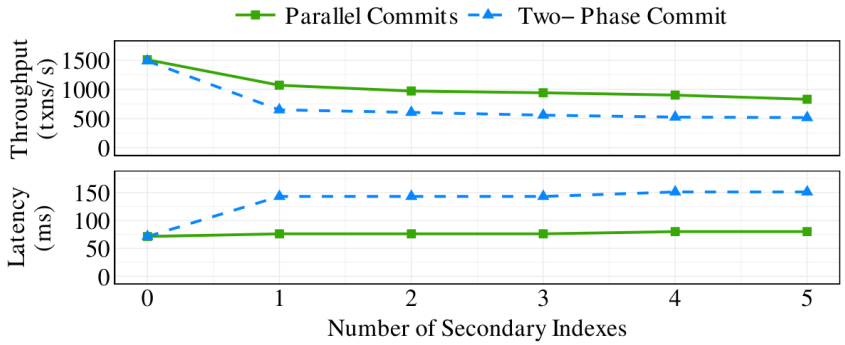

Figure 2: Performance impact of Parallel Commits

Figure 2 (paper): Parallel Commits sustains higher throughput and flatter latency than classic two-phase commit as the number of secondary indexes (and thus Ranges touched) grows.

5 Contribution 3: Controlling Where Data Lives

CockroachDB exposes several **data placement policies**, letting users trade off latency, compliance, and fault tolerance.

Policy	Behavior	Trade-off
Geo-Partitioned Replicas	All replicas of a partition pinned to one region	Fastest local reads/writes; unavailable if that region fails
Geo-Partitioned Leaseholders	Leaseholder pinned to primary region; other replicas spread across regions	Survives regional failure; slower cross-region writes
Duplicated Indexes	A read-heavy index is fully copied into every region	Fast local reads everywhere; higher write cost, more storage

Definition: Follower Reads

A mechanism letting non-leaseholder replicas serve historical (slightly stale) reads directly, using a **closed timestamp** — a running promise from the leaseholder that no future write will land at or before a given time. This spreads read traffic across all replicas, not just the leaseholder.

6 The SQL Layer: Query Optimizer, Execution, and Schema Changes

Everything discussed so far lives below the SQL layer, in the "Transactional KV" abstraction: a single, monolithic, sorted key-value store. This section covers how ordinary SQL queries actually get planned and executed on top of that abstraction.

6.1 The Query Optimizer

Definition: Cascades-Style Optimizer

CockroachDB's optimizer explores over 200 transformation rules to search the space of possible query plans and pick the cheapest one, following the same general framework (*Cascades*) used by many commercial query optimizers.

These rules are written in **Optgen**, a small domain-specific language built for CockroachDB that compiles down to Go code. A rule like `EliminateNot` simply says: replace a double logical negation with its inner expression — a *normalization* rule, since it always improves the plan and both sides are logically equivalent. Other rules are *exploration* rules (e.g. trying different join orders or join algorithms), where the optimizer keeps both the original and transformed expressions and picks whichever is estimated cheaper.

Key Concept: Distribution-Awareness

Unlike a typical single-node optimizer, CockroachDB's optimizer knows about *where data lives*. It can use a table's partitioning scheme to infer extra filters that make an index scan more selective (similar in spirit to Oracle's index skip scan, but computed statically from the schema rather than from runtime histograms). For duplicated indexes, it assigns a cost to each replica based on physical distance from the query's gateway node, so the plan naturally prefers reading the closest copy and minimizes cross-region data shuffling.

6.2 From Logical Plan to Physical Execution

Once the optimizer picks a logical plan, CockroachDB's *physical planning* stage turns it into a directed acyclic graph (DAG) of operators that can run across many nodes at once.

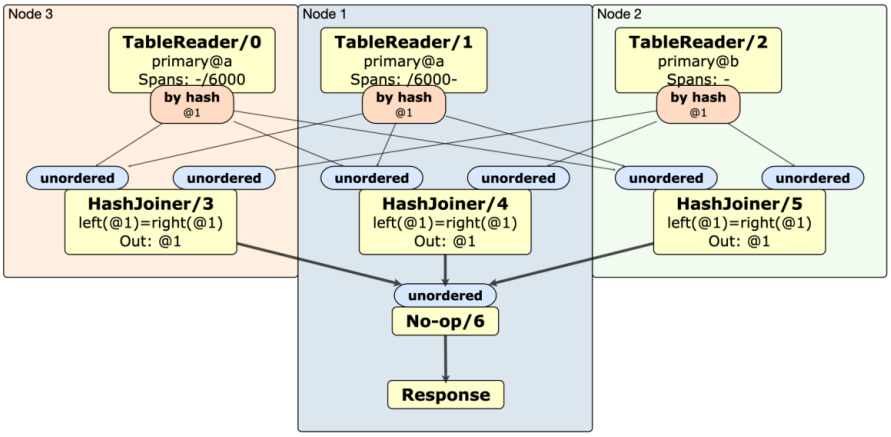

Figure 3: Physical plan for a distributed hash join

(The figure above is reproduced directly from the paper, caption included: a physical plan for a distributed hash join across two tables, `a` and `b`, on a 3-node cluster.)

In the example above, a logical scan is split into per-node **TableReader** operators, one for each Range being read. The remaining logical operators (here, hash joins) are then scheduled to run *on the same nodes* as the TableReaders that feed them, pushing computation down to where the data physically lives rather than shipping all the data to one place. CockroachDB decides whether to distribute a query at all using a simple heuristic: if a query only touches a small amount of data, it runs in gateway-only mode on a single node instead.

6.3 Two Execution Engines

Definition: Row-at-a-Time Engine

CockroachDB's primary execution engine, based on the classic *Volcano* iterator model: each operator pulls one row at a time from the operator below it. It supports every SQL feature CRDB offers (joins, aggregations, sorts, window functions), but pays a per-row interpreter overhead.

Definition: Vectorized Engine

An alternative engine, inspired by MonetDB/X100, that operates on *column-oriented batches* of rows instead of one row at a time. Because Go does not support generics with specialization, CockroachDB generates monomorphized code per data type at build time to avoid interpreter overhead. Complex operators track a *selection vector* — a compact list of which rows in a batch have not yet been filtered out — to avoid physically copying data after every filter.

The payoff is substantial: the paper reports up to two orders of magnitude speedup for individual vectorized operators, and up to 4× end-to-end speedup on TPC-H queries, for the subset of SQL that the vectorized engine currently supports.

6.4 Online Schema Changes

Key Concept: Staying Online During a Schema Change

A table must remain fully readable and writable while a schema change (e.g. adding a column or a secondary index) is being applied, even though different nodes in the cluster may briefly be running different versions of the schema. CockroachDB follows the same approach pioneered by Google's F1: decompose every schema change into a sequence of small, backward-compatible intermediate versions, and guarantee that at most two schema versions are ever active across the cluster at once.

For example, adding a secondary index requires two intermediate schema versions between the old and new schema, specifically to guarantee that the new index is being kept up to date by all writes *before* it is ever exposed to reads — otherwise a read could see a stale or incomplete index.

7 Results

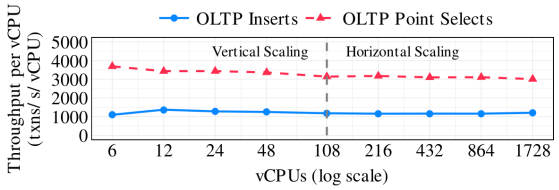

Figure 4: Maximum throughput per vCPU for Sysbench workloads with varying number of vCPUs

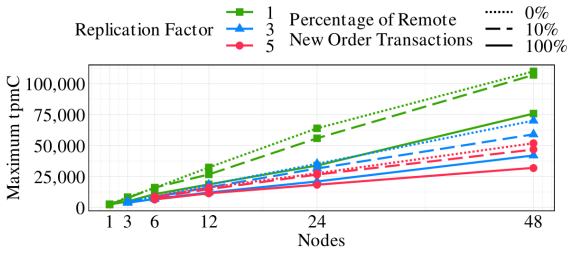

Figure 5: Maximum tpmC with varying % of remote transactions, cluster sizes, and replication factors

Figure 4 (left): throughput per vCPU stays essentially flat from 3 to 48 nodes. Figure 5 (right): overhead of cross-node coordination grows with the percentage of remote transactions and the replication factor, but throughput still scales with cluster size.

- **Scalability:** Throughput per vCPU stays nearly flat from 3 to 48 nodes on Sysbench OLTP workloads — adding machines yields close to proportional extra capacity.
- **TPC-C at scale:** At 100,000 warehouses (50 billion rows, 8 TB), CockroachDB sustains 98.8% efficiency across 81 nodes. Amazon Aurora, by contrast, drops to 7.3% efficiency at just 10,000 warehouses, because Aurora's single-master design becomes a bottleneck.
- **vs. Spanner (YCSB):** CockroachDB shows higher throughput on most workloads and lower latency at all measured percentiles under light load, with one exception — a write-heavy, highly-contended workload where CRDB's optimistic protocol scales worse.
- **Fault injection:** All four placement policies tolerate an availability-zone failure with only mild degradation. Only Geo-Partitioned Leaseholders remains available through a full *regional* failure, at the cost of higher steady-state p90 latency.

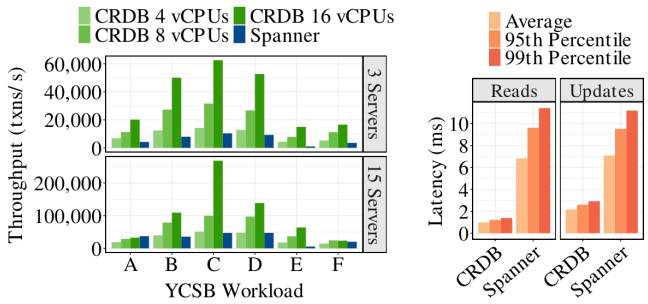

Figure 7: Throughput of CRDB and Spanner on YCSB

Figure 7 (paper): CockroachDB outperforms Spanner on most YCSB workloads and shows lower read/update latency under light load.

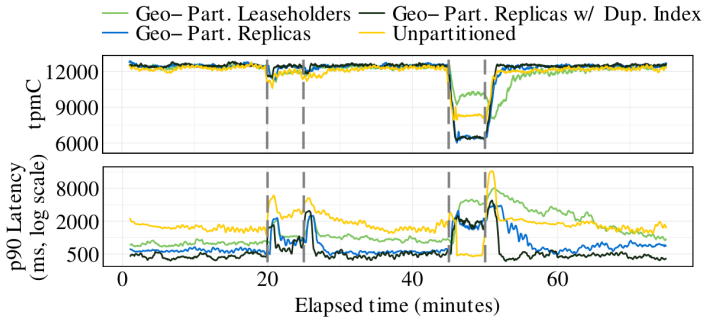

Figure 6: Multi-region cluster performance with various placement policies and AZ/region failures

Figure 6 (paper): TPC-C throughput and p90 latency across an AZ failure (∼20–26 min) and a full region failure (∼45–50 min), for each of the four placement policies.

8 Case Studies

The paper closes its evaluation with two real deployments, illustrating why the placement policies in Section 5 matter in practice (section numbers refer to the original paper).

Key Concept: A Virtual Customer Support Agent (Telecom)

A US-based telecom provider built a 24/7 virtual support agent that records customer conversation metadata in a sessions database. For financial reasons, the deployment was split across their own on-premises datacenter and AWS regions. They chose CockroachDB specifically for its strong consistency guarantees, regional fault tolerance, and support for hybrid on-prem/cloud deployments.

Key Concept: A Global Platform for an Online Gaming Company

An online gaming company processing 30–40 million financial transactions per day needed a database with strict requirements on data compliance, consistency, performance, and service availability. With a core user base in Europe and Australia and a fast-growing US base, they needed to isolate failure domains and pin user data to specific localities. To survive a full regional failure, they adopted the **Geo-Partitioned Leaseholders** policy: writes cross region boundaries to reach quorum, but reads stay local and fast.

9 Theoretical Comparison Table

System	Isolation	Clock Dependency	Concurrency Control	SQL Support
CockroachDB	Serializable + single-key linearizable	Software (NTP), loosely synced	Optimistic, timestamp-based	Full, interactive SQL
Google Spanner	Strict serializable	TrueTime (GPS + atomic clocks)	Pessimistic locking + wait-out uncertainty	Full, interactive SQL
Calvin / FaunaDB / SLOG	Strict serializable	None required	Deterministic, pre-declared read/write sets	Restricted (no ad-hoc interactive txns)
Amazon Aurora	Snapshot-style, single-master	N/A	Single-master serialization	Full SQL (Postgres/MySQL compatible)

10 Glossary

Range A ∼64 MiB contiguous shard of the sorted key space; the unit of replication and distribution in CockroachDB.

Raft A consensus algorithm used to replicate each Range consistently across multiple nodes.

Leaseholder The one replica in a Raft group authorized to serve reads and propose writes without an extra round trip.

HLC Hybrid-Logical Clock; combines physical time and a logical counter to order causally related events without needing perfectly synchronized clocks.

Uncertainty Interval A time window around a transaction's timestamp within which the real-time order of events cannot be determined with certainty.

Write Intent A provisional MVCC value written by an in-progress transaction, later resolved to committed or discarded.

Parallel Commits An optimization that replicates a transaction's "staging" commit marker in parallel with its final writes, saving a round of consensus.

Read Refresh A check that lets a transaction advance its timestamp without a full restart, by confirming previously read data hasn't changed.

Geo-Partitioning Pinning specific rows (partitions) or their leaseholders to specific geographic regions.

Follower Read A read served by a non-leaseholder replica using a closed timestamp guarantee.

Closed Timestamp
: A leaseholder's running promise that it will accept no more writes at or before a given timestamp.

TrueTime
: Google Spanner's hardware-based (GPS + atomic clock) global clock service, which CockroachDB deliberately avoids depending on.

Optgen
: The domain-specific language used to write CockroachDB's query-optimizer transformation rules; compiles to Go.

Cascades Optimizer
: A query-optimization framework that interleaves normalization (always-better) and exploration (multiple candidate) rules, picking the lowest-cost plan found.

Vectorized Execution
: An execution strategy that processes column-oriented batches of rows at once instead of one row at a time, reducing per-row interpreter overhead.

Selection Vector
: A compact list, used by the vectorized engine, of which rows in a batch have not yet been filtered out — avoids physically copying data after every filter.

11 Frequently Asked Questions

Q: Is CockroachDB just sharded Postgres?
No. Sharding (splitting into Ranges) is automatic and transparent, and cross-shard transactions get full ACID/serializable guarantees natively, unlike manually sharded systems where cross-shard transactions are often unsupported or fragile.

Q: If clocks can drift, how can transactions still be correct?
Correctness *within* a Range is enforced by Raft, which does not depend on clocks at all. Clock skew can, in rare and extreme cases, cause a stale read (a violation of single-key linearizability), but two explicit safeguards — disjoint lease intervals and per-write lease-sequence checks — ensure it can never violate transaction isolation. If clock skew exceeds 80% of the configured bound, the offending node self-terminates.

Q: Why not just use Spanner's TrueTime approach?
TrueTime requires specialized hardware (GPS receivers, atomic clocks) installed in every datacenter, which effectively ties you to one cloud provider. CockroachDB trades a small amount of theoretical strictness for the ability to run on any commodity cloud.

Q: Why did CockroachDB remove Snapshot Isolation as an option?
Supporting both Snapshot and Serializable isolation safely together would have required adding pessimistic locking to *all* row updates, even for users who only wanted Serializable, just to prevent write-skew anomalies under mixed isolation levels — not worth the performance tax for a weaker isolation level with known correctness pitfalls.

Q: Does the query optimizer know about geography?
Yes — it is described as "distribution-aware." It can push filters down using partition information and, for duplicated indexes, chooses the physically closest replica to the query's gateway node as part of normal cost-based planning.

Q: Why does CockroachDB have two execution engines instead of just one?
The row-at-a-time engine supports every SQL feature but pays a per-row interpreter cost. The vectorized engine is dramatically faster (up to 100× per operator) but, at the time of writing, only supports a subset of SQL. CockroachDB uses the vectorized engine when a query qualifies and falls back to the row-at-a-time engine otherwise, trading engine complexity for performance on the common case.

Q: How can a schema change happen without taking the table offline?
By decomposing the change into small incremental steps and never letting more than two schema versions be active across the cluster simultaneously. Each intermediate version is chosen so that data stays consistent no matter which of the two active versions a given node is currently using.

12 Conclusions

CockroachDB demonstrates that a globally distributed, horizontally scalable SQL database can provide serializable transactions on ordinary commodity hardware, by combining hybrid-logical clocks and uncertainty intervals (replacing specialized hardware clocks), an optimistic concurrency protocol with read refreshes (avoiding unnecessary retries), and two consensus-saving optimizations — write pipelining and Parallel Commits. Configurable geo-partitioning policies let the same system satisfy very different compliance, latency, and fault-tolerance requirements. A distribution-aware Cascades optimizer and two execution engines (row-at-a-time and vectorized) translate ordinary SQL into physically-local, parallel execution plans, while an F1-style online schema change protocol keeps tables available throughout schema migrations. Real deployments — a telecom support platform and a global gaming platform — confirm these mechanisms hold up in production. The authors are candid that open problems remain, including disaggregated storage, serverless scaling, and further geo-aware query optimization.

Prepared by Sumit Kumar (22CS30056) — CS60113: Advanced Database Systems.